

What Prediction Can't Do

(And Why That's Where Economics Begins)

Prof. Eduardo Zambrano

ECON 470 — Microeconomics of Artificial Intelligence

Recap: Lecture 1

- AI, economically, is **cheap prediction**
- A decision = prediction + judgment + action + outcome
- We saw a complete GPT in 200 lines:
 - tokenizer → embeddings → attention → loss → training → inference
- The output is a **probability distribution** — a prediction over what comes next

Today: Gans Chapter 2 + breaking the model

Machine Learning as Prediction

Gans, Chapter 2: Advances in Machine Learning

Traditional Statistics vs. Machine Learning

Traditional Statistics

- Researcher **specifies** the model
- Estimates a small number of parameters
- Theory-driven

Machine Learning

- Algorithm **searches** over vast model space
- Optimizes thousands to trillions of parameters
- Data-driven

microGPT does exactly this — searching over 4,192 parameters to minimize prediction error.

The Training / Inference Distinction

	Training	Inference
What	Learn from data	Use the model
Cost	Expensive (data, compute, time)	Cheap (near-zero marginal cost)
Analogy	Building a factory	Running the factory
Economics	Fixed cost	Marginal cost

⚠ Important

High fixed cost, near-zero marginal cost. What other goods have this cost structure?

Software. Pharmaceuticals. Information goods. We return to this in Weeks 6–7.

What Made Modern AI Possible

1. **More data** — the internet
2. **More compute** — GPUs, cloud infrastructure
3. **Better architecture** — the attention mechanism (you saw it yesterday in `gpt()`)
4. **Scale** — more parameters + more data = better predictions

How the Model Learns: The Chain Rule

Last class we saw `loss.backward()` — one line that adjusts all 4,192 parameters. How?

You set your thermostat to x degrees. The AC uses $y = f(x)$ kilowatt-hours of electricity. Your bill is $z = g(y)$ dollars.

How much does your bill change per degree?

$$\frac{dz}{dx} = \frac{dz}{dy} \times \frac{dy}{dx} = (\text{price per kWh}) \times (\text{extra kWh per degree})$$

That's the **chain rule**: multiply the rates of change through a chain of operations. The model is a long chain of operations from input to loss.

Backprop in Action

Let's run it. Here is Karpathy's `Value` class and a tiny computation graph:

$L = (a \times b) + a$, with $a = 2$, $b = 3$.

```
1 class Value:
2     __slots__ = ('data', 'grad', '_children', '_local_grads')
3     def __init__(self, data, children=(), local_grads=()):
4         self.data, self.grad = data, 0
5         self._children, self._local_grads = children, local_grads
6     def __add__(self, other):
7         other = other if isinstance(other, Value) else Value(other)
8         return Value(self.data + other.data, (self, other), (1, 1))
9     def __mul__(self, other):
10        other = other if isinstance(other, Value) else Value(other)
11        return Value(self.data * other.data, (self, other), (other.data, self.data))
12    def __radd__(self, other): return self + other
13    def __rmul__(self, other): return self * other
14    def backward(self):
15        topo, visited = [], set()
16        def build_topo(v):
17            if v not in visited:
18                visited.add(v)
19                for child in v._children: build_topo(child)
20                topo.append(v)
21        build_topo(self)
22        self.grad = 1
23        for v in reversed(topo):
24            for child, lg in zip(v._children, v._local_grads):
25                child.grad += lg * v.grad
26
27 v = Value(2.0)
```

```
L = 8.0
dL/da = 4.0 (b + 1 = 3 + 1, because a flows through two paths)
```

What Machine Learning Optimizes

```
1  """
2  The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
3  This file is the complete algorithm.
4  Everything else is just efficiency.
5
6  @karpathy
7  """
8
9  import os          # os.path.exists
10 import math         # math.log, math.exp
11 import random      # random.seed, random.choices, random.gauss, random.shuffle
12 random.seed(42)    # Let there be order among chaos
13
14 # Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
15 if not os.path.exists('input.txt'):
16     import urllib.request
17     names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
18     urllib.request.urlretrieve(names_url, 'input.txt')
19 docs = [line.strip() for line in open('input.txt') if line.strip()]
```

- The model has an **objective function** imposed by its designers
- It does **prediction**. Humans supply **judgment**.
- These are **complements**, not substitutes.

This is the key conceptual move for the entire course.

Cross-Entropy Loss: The Intuition

The model predicts a probability for each possible next character. The loss measures: **how surprised was the model by the correct answer?**

$$\text{loss} = -\log P(\text{correct character})$$

- Model assigns **90%** to the right answer $\rightarrow \text{loss} = -\ln(0.9) = 0.11$ (barely surprised)
- Model assigns **10%** to the right answer $\rightarrow \text{loss} = -\ln(0.1) = 2.30$ (very surprised)
- Model assigns **1/27** (random guessing) $\rightarrow \text{loss} = -\ln(1/27) = 3.30$ (maximally clueless)

growingswe.com/blog/microgpt

Inference and Temperature

Remember **temperature = 0.5** from the inference code? The model samples from $P(\text{token}_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$, where z_i are the raw logits and T is the temperature.

Temperature	Effect on distribution	Economic analogy
$T \rightarrow 0$	All mass on highest z_i — deterministic	Exploitation
$T = 1$	Standard softmax — learned probabilities	Neutral
$T \rightarrow \infty$	Uniform over all tokens — pure	Exploration

Explore microGPT Interactively

[microGPT Interactive: See the full pipeline in your browser](#)

“Now that we’ve seen how a GPT works, let’s **break** it.
Not to be destructive — but because what breaks reveals
what the model actually is and what it **isn’t**.
And each failure points to an **economic question**.”

Five Experiments

Breaking a 200-line GPT to understand bigger models

Inspired by [Narnaiezzsshaa \(2026\)](#)

Experiment 1: Baseline

What did the model learn?

After training (loss: 3.3 $\rightarrow$ ~2.37), generate 100 names:

akalya	brana	anana	eralia	jasha
anene	marin	ielā	arylan	anelan
aman	sazyen	azena	alelen	amana
ana	aniel	tonlin	arannn	shai

- Not real names — but they **sound like** names
- 64 out of 100 were **novel** (not in the training data) — the model generalized
- 36 matched real names — it also captured common patterns exactly
- Average length: 5.1 characters (training data average: ~6)

Economic point

Recall from last class: loss 3.3 = $-\ln(1/27)$ means random guessing (3.7% chance per character). Loss 2.37 means $e^{2.37} \approx 10.7$ effective choices (9.3% chance). That's a **+5.6 percentage point** gain — the model is **2.5× more likely** to pick the right character. This is what Gans means by “prediction reduces uncertainty,” and you can measure it.

Experiment 2: Data Drift

What if we feed it reversed names?

Train on: emma → amme, olivia → aivilo, ava → ava

Forward model generates:

dalern, kayen, arelia, raych, daam,
javi, catar, zilah

Reversed model generates:

eiilel, netel, aylera, naylim, adh,
nayl, aita, nyrade

The model has **no concept of “correctness.”** It learned the reversed distribution with no complaints, no warnings, no errors.

Economic point

The model can't detect biased or corrupted data. This is the **data quality problem** — and it's why Gans devotes Chapters 8–9 to the economics of data supply. Someone must exercise *judgment* about what data to use.

Experiment 3: Context Window

What happens past the model's memory limit?

The model was trained with a context window of **16 tokens**.

Force it to generate **32 tokens** (no early stopping).

The **|** marks position 16 — the edge of the context window:

```
raisannnahahane | lahaytozaraharh  
avaslilnaeelaora | naiarliannabbbin  
saklanahannlahen | ahahahelennahyna  
toyneonnmiahhann | ahannahahahaiahuta  
abratonnaynelene | reynelnahanilelo
```

- Left of **|**: recognizable name-like fragments
- Right of **|**: degrades into repetitive **ahahah** patterns
- The architecture **constrains** what can be predicted — more training won't fix this

Context Window: The Deeper Point

The model doesn't **know** it's past its limit.

It doesn't flag uncertainty or say "I don't know."

It just produces **garbage with the same confidence.**

Economic point

Every AI system has architectural limits. Organizations adopting AI need to understand whether the *task* falls within the AI's capacity — not just whether it's accurate on training data. The judgment of **when to trust** a prediction is valuable.

Experiment 4: Weight Perturbation

What if we slightly nudge the parameters?

Take the trained model. Add Gaussian noise to every parameter. Try three scales:

Noise scale	Overlap with clean model
0.01 (tiny)	14% names in common
0.05 (small)	4% names in common
0.10 (modest)	0% names in common

Clean model: `asan, mani, kana, elina, leran, marel, arelan`

After noise 0.1: `asari, alizi, alie, anae, arahi, arai, arale`

Important

The model's knowledge lives on a **narrow ridge** in parameter space. Even tiny perturbations degrade it; modest noise annihilates it completely.

Weight Perturbation: Three Economic Implications

1. **Intellectual property:** The value is entirely in specific parameter values. Copy the numbers → copy the model. Perturb them → destroy it. AI IP is fundamentally different from a patented process.
2. **Robustness:** Why use a simple rule when AI is available? Because rules don't **catastrophically fail** when the environment shifts. A rule like “reject loans below 650” is robust. The AI model might not be. (Gans, Ch. 4)
3. **Security:** Access the parameters + add noise = sabotage. An adversarial risk without a clean analogue in traditional production.

Experiment 5: Prompting the Model

First, let's see that prompting works.

Feed the model **[BOS]** **m a r** and let it continue:

marian	marin	marana	marian	maria
maria	marian	mareia	maris	mari

The model sees **mar** and continues with plausible completions: Maria, Marian, Marin. This is prompting — give it a prefix, it predicts what comes next. So far, so good.

Experiment 5: “Long”

Now let’s try an instruction. Feed the model **[BOS] l o n g**:

longia longa longi longa longan

longan longh longi longa longin

We asked for a long name. The model treated **long** as a name prefix and completed it — **longia**, **longa**, **longan**. Most are 5–6 characters, same as baseline. The word “long” meant nothing as an instruction.

What If We Add a Boundary?

OK, but what if we add [BOS] after `long` to separate the “instruction” from the name? Feed the model [BOS] `l o n g` [BOS]:

an	a	a	a	a
an	a	a	ee	a

Average length: **1.1 characters**. Unprompted average: **5.0 characters**.

We asked for long names and got the *shortest possible* outputs. The model saw [BOS] `l o n g` as a previous “name,” but it was never trained on sequences of two names. It’s **out of distribution** — the attention is processing a context it has never encountered, and the result is degraded, not obedient.

Why Can't It Follow Instructions?

The model does one thing: $P(\text{next token} \mid \text{all previous tokens})$.

“Long” is an **instruction** — a goal, a constraint. The model has no mechanism for goals. It only has a mechanism for prediction. It treats every input the same way: as a prefix to complete.

When ChatGPT *appears* to follow instructions, that's because RLHF (Reinforcement Learning from Human Feedback) shifted the distribution so that instruction-following outputs are more *probable*. But probable isn't guaranteed. The mechanism is still prediction, not execution of a decision procedure.

This is the most important experiment for our course.

Prediction $\neq$ Judgment

The model is excellent at **prediction** — it learned the statistical distribution of names.

It cannot incorporate **judgment** — external criteria about what constitutes an acceptable output.

We tried to give it an instruction and it treated it as just another sequence of tokens. Instructions require understanding. The model only has statistics.

Why Judgment Is a Complement

Better prediction doesn't **reduce** the need for judgment — it **increases** it, because you're using prediction in more decisions and each one requires criteria for what a good outcome looks like.

This is why production AI systems have layers of filtering, guardrails, and human review **on top of** the model.

Those aren't patches. They're the **judgment layer** that prediction, by its nature, cannot provide.

The economic question: **who provides that judgment, what does it cost, and how should it be organized?**

Synthesis

Mapping Failures to Economics

What we saw	Economic concept	Course topic
Predicts probability distributions	AI as cheap prediction	Ch. 1–3
Training costly, inference cheap	High-fixed, low-marginal cost	Pricing (Wk 6–7)
Learns whatever data it gets	Economics of data supply	Data (Wk 5)

Mapping Failures to Economics (cont.)

What we saw	Economic concept	Course topic
Architectural limits	Demand for prediction	Demand (Wk 2–3)
Perturbations destroy knowledge	IP, robustness, security	Policy (Wk 8–10)
Cannot enforce constraints	Prediction $\neq$ judgment	Complements (Wk 3)

Every failure mode generates an economic question. That's the course.

For the next eight weeks, we build the **formal frameworks** to analyze these questions.

For Next Class

Read: Gans, Chapter 3 (The Value of Prediction)

Next week we **formalize** the value of prediction.

Gans builds a model where an agent faces a decision under uncertainty and shows exactly how better prediction changes the optimal decision.

You now have an intuition for what “better prediction” means mechanically.

Next week we make it **precise**.

Reminder: Start browsing the Digitalist Papers for your presentation.

digitalistpapers.com/volume2